Transcrição

Vamos definir o que todo filme precisa ter.

Criando a estrutura de um filme

No arquivo `Filme.php`, criaremos uma estrutura que ainda não conhecemos, chamada `Filme`. Quando criamos algo que delimita um bloco, precisamos usar chaves (`{}`), então vamos abrir e fechar chaves. Dentro delas, definiremos alguns valores. Antes de continuarmos, é importante lembrarmos que, no início do arquivo, precisamos colocar a abertura da tag PHP (`<?php`), porque tudo que escrevermos nesse código será em PHP.

```
<?php

Filme {

}
```

COPIAR CÓDIGO

Dentro dessas chaves, na linha 4, após identarmos, definiremos quais são os valores que todo filme precisa ter. Primeiro, todo filme precisa ter um **nome** ou **título**, como você preferir. Eu chamarei de `nome`, então criarei a variável `$nome`. Também precisa ter um **ano de lançamento** e um **gênero**, então criaremos essas variáveis.

```
<?php

Filme {
    $nome;
    $anoLancamento;
    $genero;
}
```

COPIAR CÓDIGO

Voltando para o nosso arquivo `funcoes.php` para descobrir o que mais nosso filme precisa. Além do nome, ano e gênero, a função `criaFilme()` tem a variável **nota**, portanto, vamos adicioná-la ao nosso modelo de `Filme` também.

Arquivo `Filme.php`

```php
<?php

Filme {
    $nome;
    $anoLancamento;
    $genero;
    $nota;
}
```

COPIAR CÓDIGO

Salvamos esse arquivo e agora vejamos o que acontece. Como temos algumas extensões do PHP no nosso Visual Studio Code, às vezes ele apresenta alguns avisos quando salvamos, e foi exatamente o que aconteceu. No caso, a variável `$nome` ficou sublinhada em vermelho.

Analisando o erro

Ele indica um erro de sintaxe, porque escrevemos um ponto e vírgula que ele não estava esperando. Na verdade, o que acabamos de criar não faz muito sentido. O PHP se perdeu no que estamos fazendo, porque criamos o `Filme` sem dizer se ele uma função ou outra coisa.

O que estamos definindo é um modelo (*template*) dizendo o que todo filme que criarmos precisa ter. Todo filme que

criarmos, ou seja, toda variável que for um `Filme`, precisa ter nome, ano de lançamento, gênero e nota.

O **nome para estrutura que modela algo**, criando esse *template*, é **classe**. Estamos criando a **classe Filme, ou seja,** `class Filme`.

```
<?php

class Filme {
    $nome;
    $anoLancamento;
    $genero;
    $nota;
}
```

COPIAR CÓDIGO

Entendendo sobre a classe (`class`)

Uma **classe** nada mais é do que um novo tipo, no caso, um tipo próprio que estamos criando, chamado `Filme`. Assim como os tipos como string e inteiro, criamos o tipo `Filme`.

Porém, os tipos como inteiro, string, booleano e os outros são chamados **tipos primitivos**. porque têm apenas **um único valor**, já **o tipo** `Filme` **é mais complexo,**

porque possui vários valores. Vamos nos adaptando com isso ao decorrer do curso, mas, basicamente, estamos criando um novo tipo, chamado `Filme`.

Ao salvarmos a mudança da linha 3, onde escrevemos `class Filme {`, ainda temos o erro. Isso porque não podemos simplesmente criar uma variável na classe sem explicitar que estamos criando algo que faz parte de um modelo do filme.

Temos algumas opções para fazermos isso, usando **palavras reservadas**. Ainda sem explicar muito detalhes, utilizaremos uma palavra que nos permite **acessar as variáveis do tipo** `Filme`, como o `nome` e o `anoLancamento`. Para isso, **usaremos a palavra** `public` **antes de cada variável**.

```php
<?php

class Filme {
    public $nome;
    public $anoLancamento;
    public $genero;
    public $nota;
}
```

COPIAR CÓDIGO

Quando salvamos, não temos mais nenhum erro!

Declarando os tipos das variáveis

Antes de, efetivamente, utilizarmos esse novo tipo, precisamos **explicitar os tipos de cada uma das variáveis dele**. Assim como nos tipos de parâmetros e tipo de retorno de funções, dizer os tipos do que tem dentro do nosso filme também é **opcional**, mas

`anoLancamento` é um inteiro, o `genero`, por enquanto, também será uma string, e a `nota` será um float. Poderíamos questionar o tipo dessas duas últimas variáveis e pensar em outras abordagens, mas esses tipos nos atendem bem por enquanto e é o que temos utilizado até agora.

```php
<?php

class Filme {
    public string $nome;
    public int $anoLancamento;
    public string $genero;
    public float $nota;
}
```

COPIAR CÓDIGO

Feito isso, voltaremos para o arquivo `funcoes.php`, porque, na função `criaFilme()`, ao invés de retornarmos um

`array` associativo, retornemos um `Filme`. Essa função precisa retornar uma variável do tipo `Filme` no lugar do array.

Para isso, dentro da função criaremos uma variável `$filme` e vamos pensar em como criamos um filme. Temos a classe `Filme` e precisamos criar uma variável com esse tipo. Para isso, utilizaremos outra **palavra reservada, chamada `new` (novo)**. No caso, `new Filme()`.

Arquivo `funcoes.php`

```
// código omitido

function criaFilme(string $nome, i
{
    $filme = new Filme()

    return [
        'nome' => $nome,
        'ano' => $anoLancamento,
        'nota' => $nota,
        'genero' => $genero
    ];
}
```

COPIAR CÓDIGO

Criamos uma variável do tipo `Filme()`. Os parênteses que escrevemos, nesse momento, são **opcionais**, mas vamos nos acostumar a sempre utilizá-los. Em breve você entenderá o motivo para isso.

Agora que temos a variável do tipo `Filme`, queremos acessar, de alguma forma, o nome dele. Em um array associativo, utilizamos colchetes e passamos string `'nome'`, por exemplo, `$filme['nome']`. Porém, para uma **variável criada a partir de uma classe**, utilizamos outro operador para acessar as variáveis, o **operador seta** (`->`), que é um traço (`-`) seguido do sinal maior que (`>`).

Utilizando o operador de seta

A partir dessa seta, podemos acessar o `nome`, escrevendo `$filme->nome = $nome;`. Assim atribuímos à variável `nome` o nome que recebemos por parâmetro. Faremos a mesma coisa com o ano de lançamento, o gênero e a nota.

Arquivo `funcoes.php`

```
// código omitido

function criaFilme(string $nome, i
```

```
{
    $filme->nome = $nome;
    $fi1me->anoLancamento = $anoLa
$filme->genero = $genero;
$filme->nota = $nota;

    return [
        'nome' => $nome,
        'ano' => $anoLancamento,
        'nota' => $nota,
        'genero' => $genero
    ];
}
```

COPIAR CÓDIGO

Atenção: Precisamos garantir nossa classe de `Filme` tenha os mesmos parâmetros que passamos para função.

Abrindo o arquivo `Filme.php`, percebemos que temos todos essas variáveis, escritas da mesma forma que passamos para a função:

- `nome`;
- `anoLancamento`;
- `genero`;
- `nota`.

Os nomes que passamos nos parênteses de `criaFilme()` e o nome da variável que

damos dentro da função, por exemplo `$nome`, não importa. O que precisamos verificar é o nome que passamos depois do operador seta, que precisa ser igual ao da classe.

Se passamos `$filme->nome`, nossa classe `Filme` precisa ter um parâmetro `nome`. Outro exemplo, se em `Filme.php` mudamos de `anoLancamento`, para `ano`, no `funcoes.php` precisamos passar `$filme->ano = $anoLancamento`. O nome dentro da função pode continuar `$anoLancamento`, mas **o nome após a seta tem que ser idêntico ao da classe**. No momento, manteremos como `anoLancamento`.

De volta ao arquivo `funcoes.php`, na função `criaFilme()`, já criamos o nosso filme e atribuímos os valores do nosso filme. Agora, podemos retornar essa variável `$filme`, codando `return $filme;`. Com isso, podemos apagar da linha 28 a 33, onde estava o retorno do array associativo.

Arquivo `funcoes.php`

```
// código omitido

function criaFilme(string $nome, i
{
    $filme->nome = $nome;
```

```
    $fi1me->anoLancamento = $anoLa
$filme->genero = $genero;
$filme->nota = $nota;

    return $filme;
}
```

COPIAR CÓDIGO

Agora, abriremos o arquivo `index.php`, onde temos a função `criaFilme()`, na linha 38. Nossa `$filme` não é mais um array associativo, então, em todo `index.php`, ao invés de acessarmos através da sintaxe com colchetes (`[]`), usaremos o **operador de seta**. Por exemplo, na linha 45 mudaremos `echo $filme["ano"];` para `echo $filme->anoLancamento;`, assim como fizemos dentro da função.

Mudanças no código do `index.php`

- Linha 45: `echo $filme->anoLancamento;`
- Linha 53: `var_dump($filme->nome);`
- Linha 54: `$posicaoDoisPontos = strpos($filme->nome, ':');`

- Linha 57: `var_dump(substr($filme['nome'], 0, $posicaoDoisPontos));`

Testando as mudanças e corrigindo erros

Após fazermos essas mudanças em todas as aparições de `$filme[]` para `$filme->`, usando o mesmo nome de variável da classe `Filme`, já temos o necessário para continuar a execução. Então, abriremos o terminal, onde digitaremos `php index.php`, para rodarmos esse arquivo que alteramos.

Quando executamos, recebemos a notificação de um **erro de divisão por zero** (***Division by zero***), porque não nos lembramos de passar as **notas** para esse filme na hora da execução. Portanto, vamos executar mais uma vez, mas agora passando as notas: `php index.php 10 8 7 5.8 e 9`

Agora tivemos **outro erro**, indicando que a classe `Filme` não foi encontrada ("*Uncaught Error: Class "Filme" not found*"). Realmente, no arquivo `index.php`, **não definimos a classe** `Filme` em lugar nenhum.

Assim como importamos o `funcoes.php`, no começo do código, precisamos importar o

`Filme.php`. Faremos isso logo na primeira linha, abaixo abertura da nossa tag de abertura `<?php`.

```
<?php

require __DIR__ . "/src/Modelo/Filı
require __DIR__ . "/src/funcoes.phı

// código omitido
```

COPIAR CÓDIGO

Agora, quando executamos o `php index.php 10 8 7 5.8 e 9` de novo, nosso código é executado sem erro nenhum. A diferença é que agora temos um pouco mais de organização. Sabemos tudo o que um filme precisa para efetivamente ser um filme, então as coisas começam a ficar mais organizadas.

Apesar de estar melhor, existem muitos códigos que podemos organizar mais. Portanto, no próximo vídeo, entenderemos um pouco melhor **a teoria do que acabamos de fazer**.

Descobriremos os **processos** por trás das mudanças que fizemos e revisaremos alguns **conceitos** que já aprendemos. Esperamos você no próximo vídeo para desvendarmos

alguns dos segredos que já até implementamos.